

Grading Criteria

Form (30%) <i>To time? Verbal speed & clarity? Body posture? Engagement with audience?</i>	Moderation (30%) <i>Integrates questions well? Pushes forward discussion? Good summary? Strengths and weaknesses of paper?</i>	Practice Lecture (30%) <i>Listen attentively to the lecture? Engage in small coding exercises?</i>
Content (50%) <i>Structure/storyline? Main points? Paper connections? Valid conclusions?</i>	Questions (70%) <i>One question per paper (two questions per class) should be submitted at least one hour before the class during which the paper will be presented. (However, students are permitted to submit questions late (but before the discussion), up to two occurrences, without facing any penalties)</i>	Discussion on your favorite papers (35%) <i>Each person has 5 minutes to present their favorite paper. Is your presentation clear? Explain clearly why you chose it and what you like about it?</i>
Answers (20%) <i>Good answers to questions? Knowledgeable?</i>		Brainstorming (35%) <i>Actively participate in the discussion? Contribute your own ideas or opinions?</i>

2x Presentations
(50% of grade)

Discussion
(25% of grade)

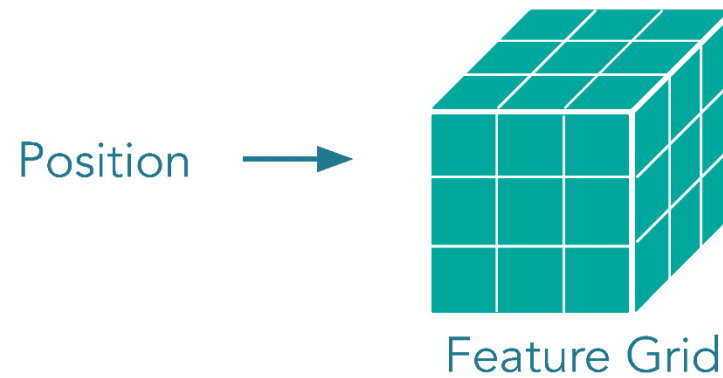
Other Activity Participation
(25% of grade)

Fast Training of Neural Radiance Fields

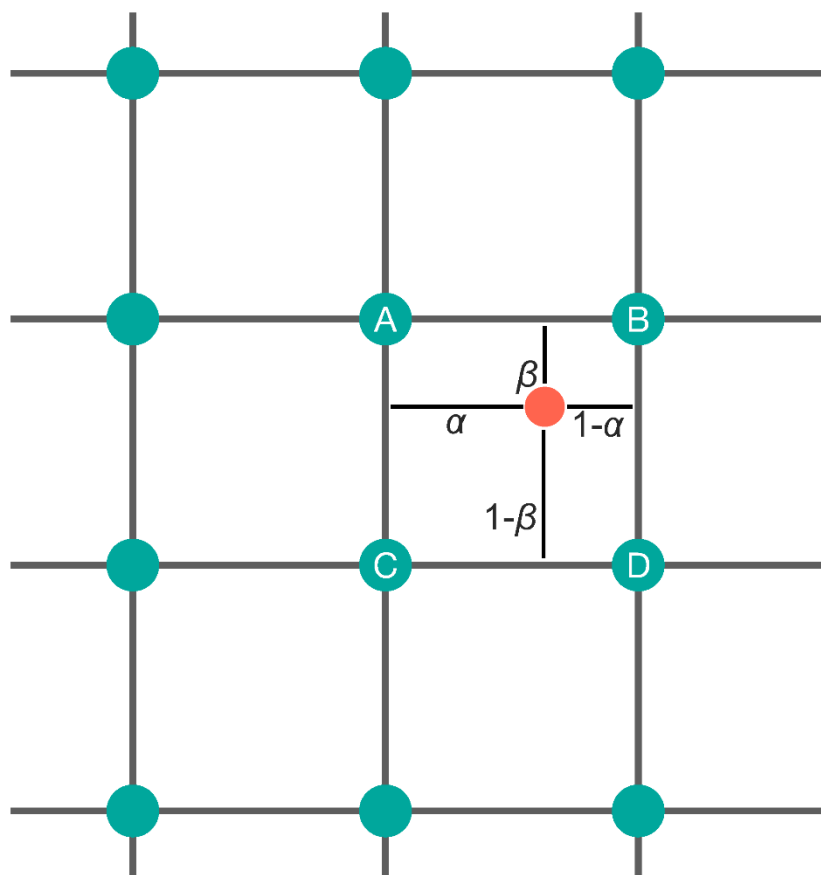
Lingjie Liu



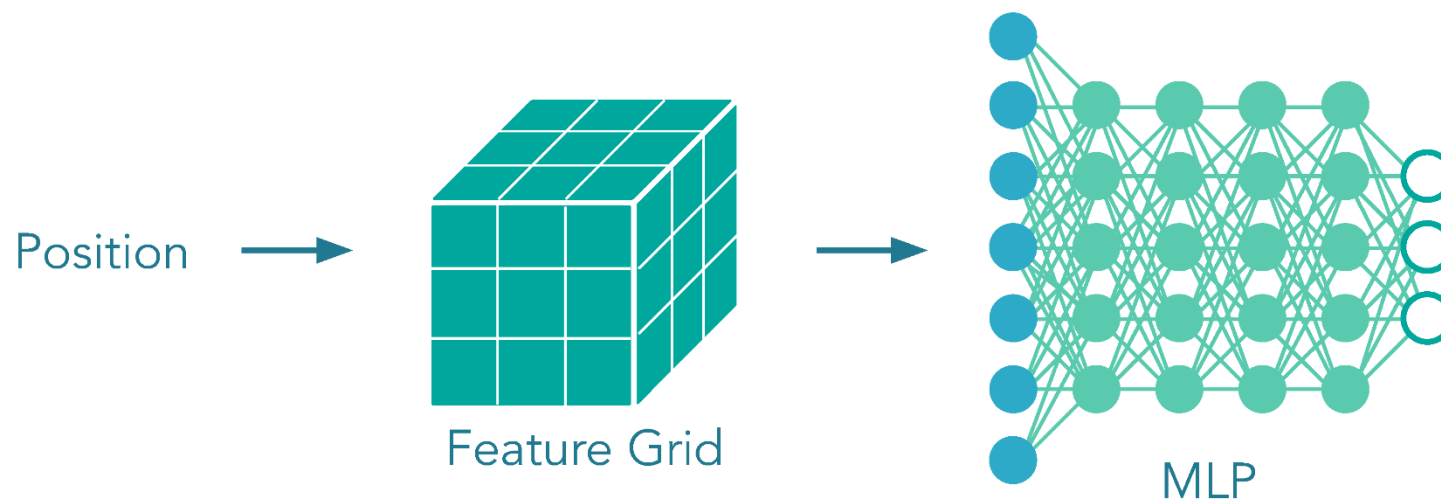
Feature Grids as MLP Encodings



Make continuous via interpolation



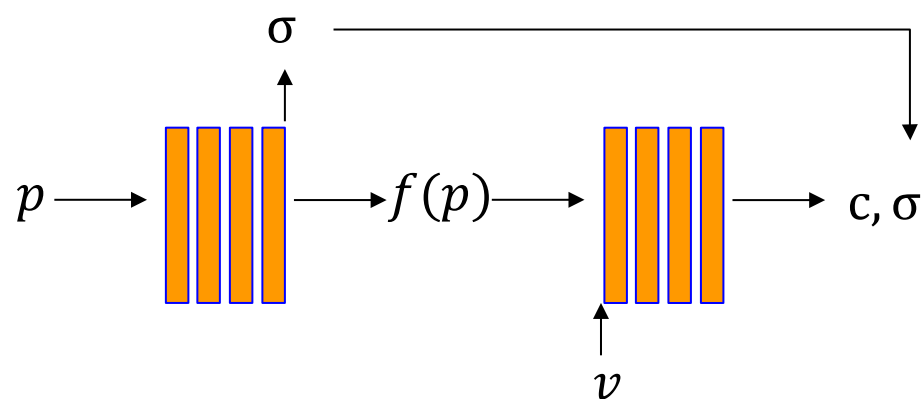
Feature Grids as MLP Encodings



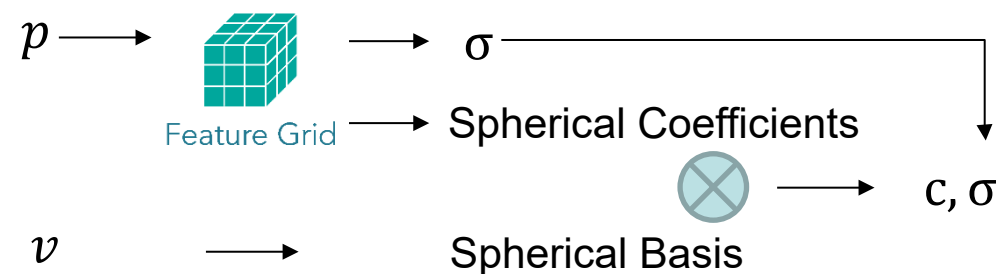
Plenoxels

- Split NeRF's neural network into two separate networks: pose-dependent network and view-dependent network.

$$c(v; \mathbf{k}) = S \left(\sum_{\ell=0}^{\ell_{\max}} \sum_{m=-\ell}^{\ell} k_{\ell}^m Y_{\ell}^m(v) \right)$$

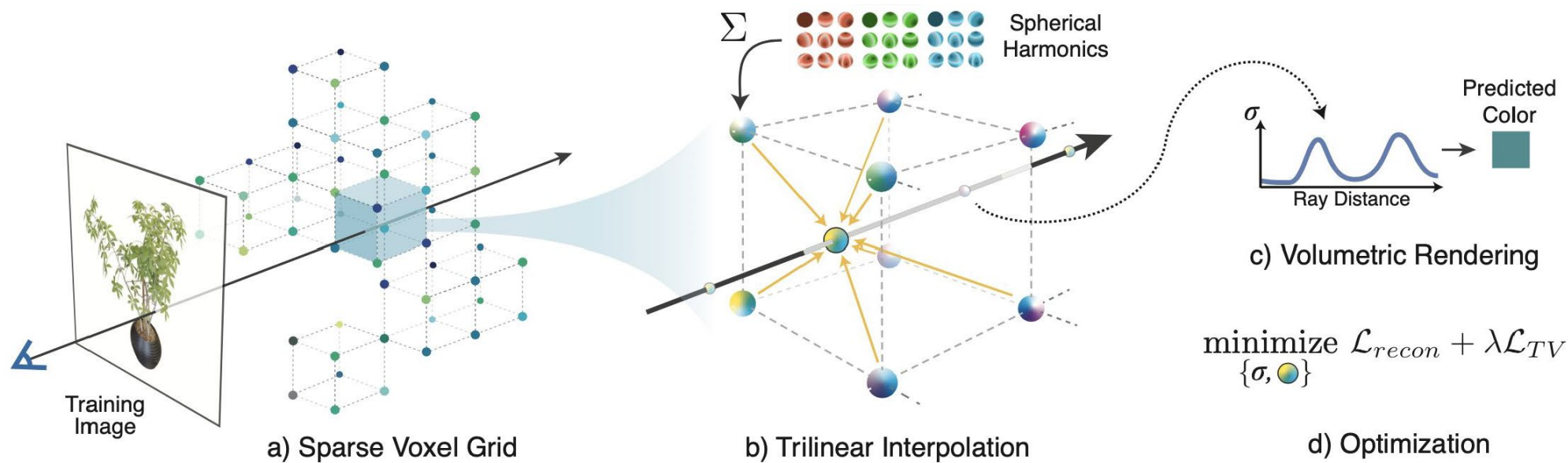


Original NeRF Network

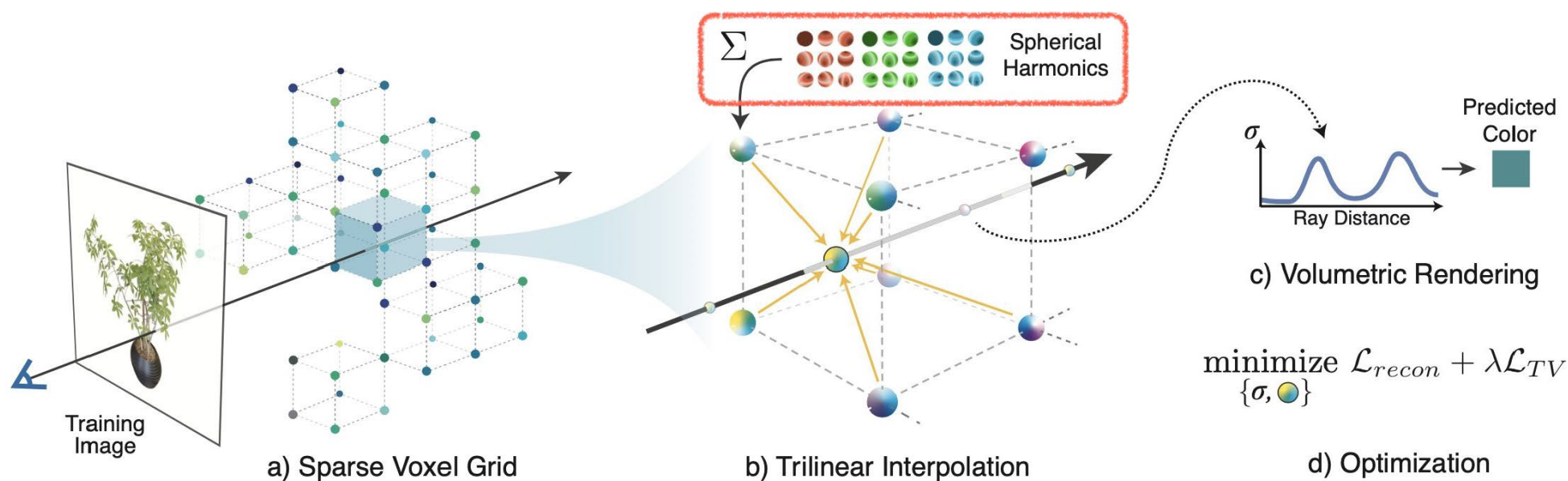


Network Design in [Yu et al. 2021]

MLPs are not required...



MLPs are not required...



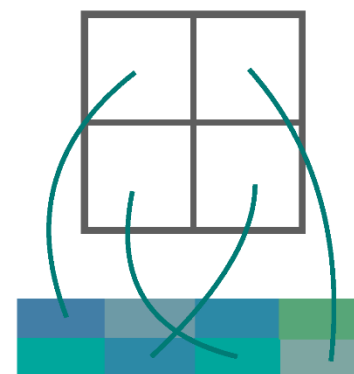
Compression Techniques



Sparsity

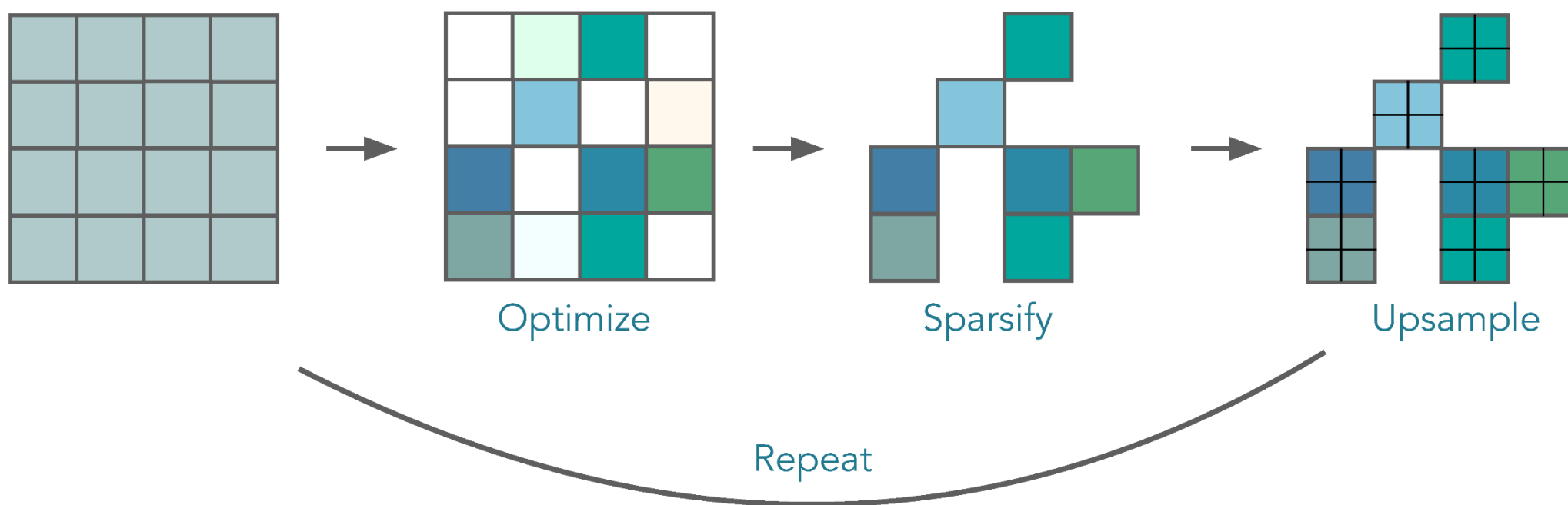


Low Rank



Dictionary

Sparsity



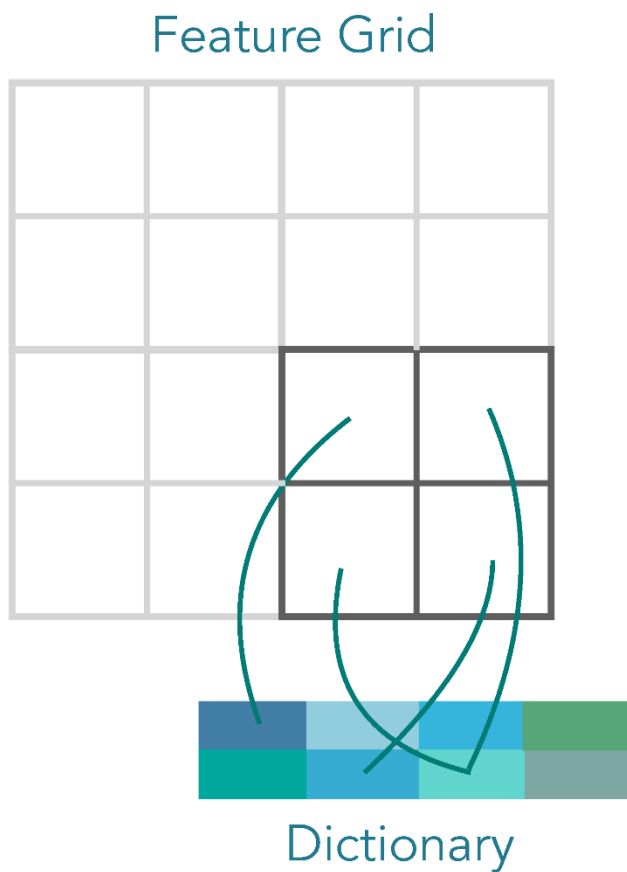
Lui et al. 2020, Neural Sparse Voxel Fields
 Yu*, Friedovich-Keil* et al. 2021, Plenoxels: Radiance Fields without Neural Networks
 Sun et al. 2021, Direct Voxel Grid Optimization: Super-fast Convergence for Radiance Fields Reconstruction

Low Rank Approximations



Chen*, Xu* et al. 2022, TensorRF Tensorial Radiance Fields
Chan*, Lin*, Chan*, Nagano* et al. 2022, EG3D: Efficient Geometry-aware 3D Generative Adversarial Networks

Dictionary Methods



Feature Grid > Dictionary

Mapping with collisions

Related Work

- Plenoxels: Radiance Fields without Neural Networks, Yu et al., CVPR 2022
- Direct Voxel Grid Optimization: Super-Fast Convergence for Radiance Fields Reconstruction, Sun et al., CVPR 2022
- Instant Neural Graphics Primitives with a Multiresolution Hash Encoding, Müller et al., SIGGRAPH 2022
- TensorRF: Tensorial Radiance Fields, Chen et al., ECCV 2022
- NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis, Mildenhall et al., ECCV 2020

Fast Rendering of Neural Radiance Fields

Lingjie Liu

Background

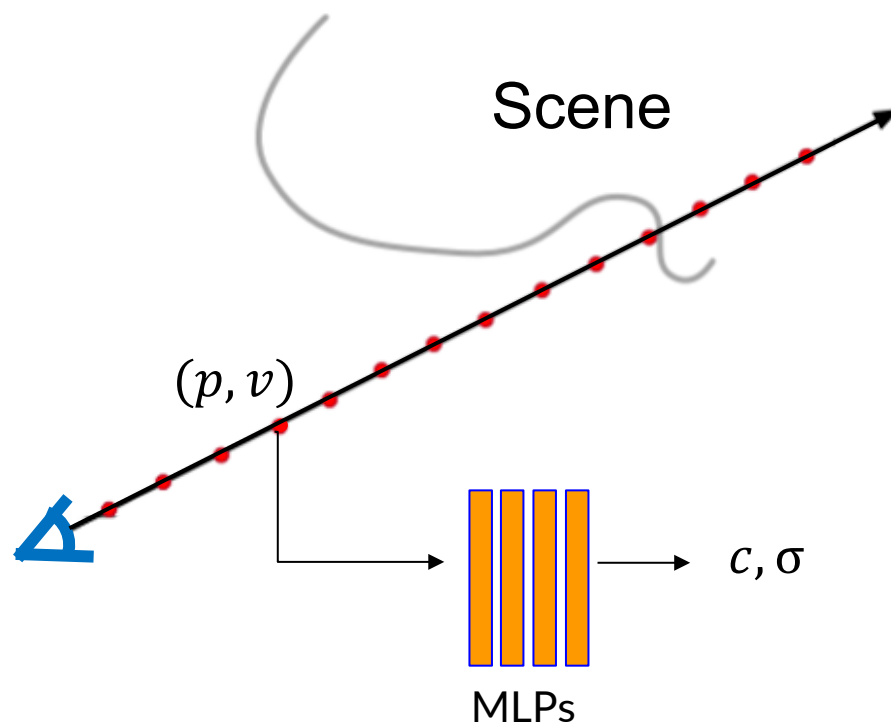


Illustration of volume rendering in NeRF

Background

- NeRF (Mildenhall et al. 2020)



Rendering speed: 100 s/frame

Image resolution: 1920x1080

To render an image at 1920x1080 pixels,
how many calls of the MLPs are needed?

$$(1920 \times 1080) \times 192 = 398,131,200$$

It takes about 100 seconds to render such
an image using an NVIDIA V100 GPU

Background

- NeRF (Mildenhall et al. 2020)

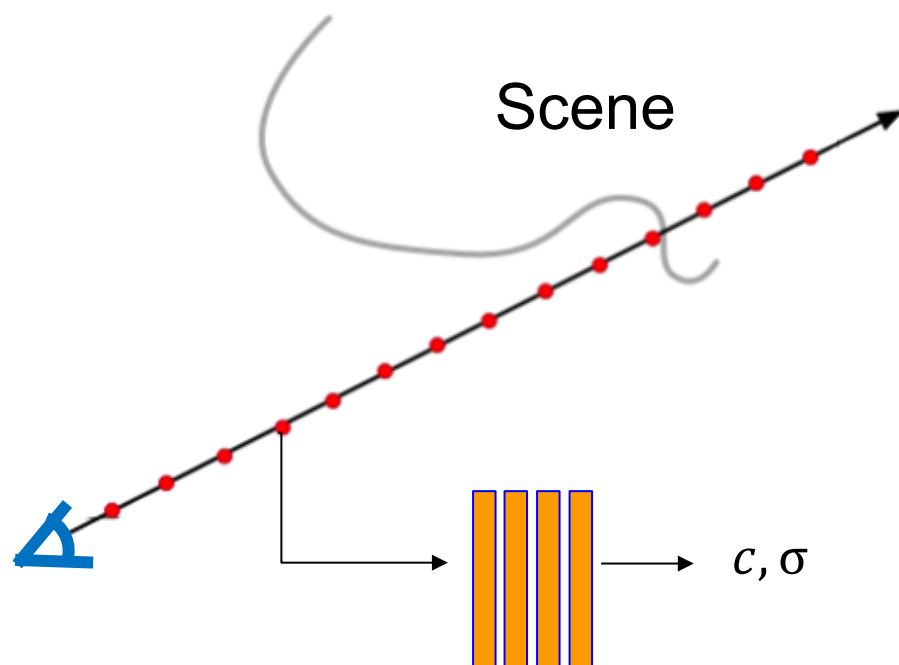


Illustration of volume rendering in NeRF

To render an image at 1920x1080 pixels, how many calls of the MLPs are needed?

$$(1920 \times 1080) \times 192 = 398,131,200$$

It takes about 100 seconds to render such an image using an NVIDIA V100 GPU

Two possible ideas to accelerate the rendering process:

1. Reduce sampling points.
2. Reduce the runtime for one pass.

Neural Sparse Voxel Fields

Lingjie Liu*, Jiatao Gu*, Kyaw Zaw Lin, Tat-Seng Chua, Christian Theobalt (* equal contribution)

NeurIPS 2020 Spotlight

Our Method -- Neural Sparse Voxel Fields (NSVF)

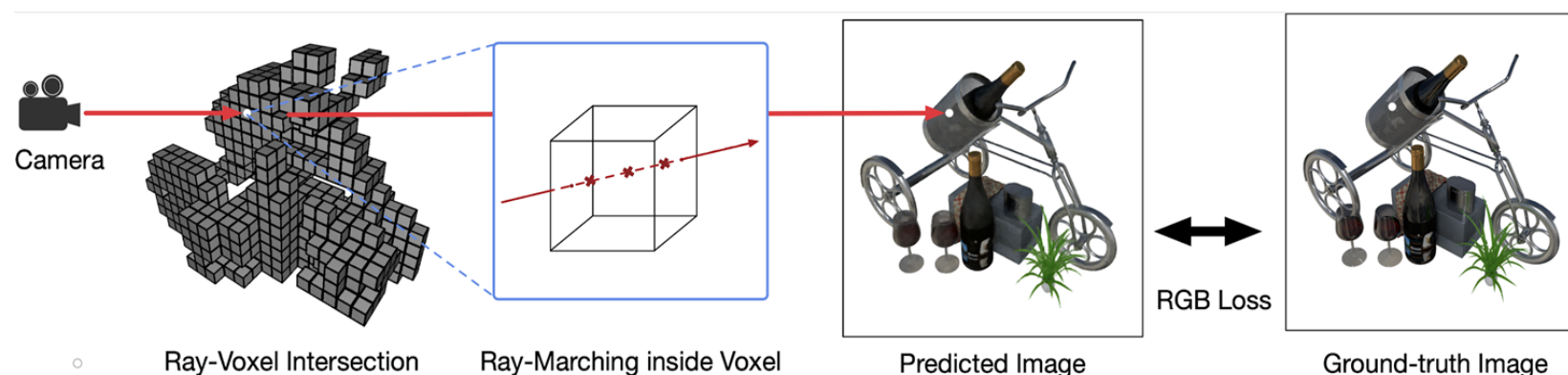
- Avoid sampling points in empty space as much as possible.
- Neural Sparse Voxel Fields (NSVF), a hybrid scene representation for fast and high-quality free-viewpoint rendering.



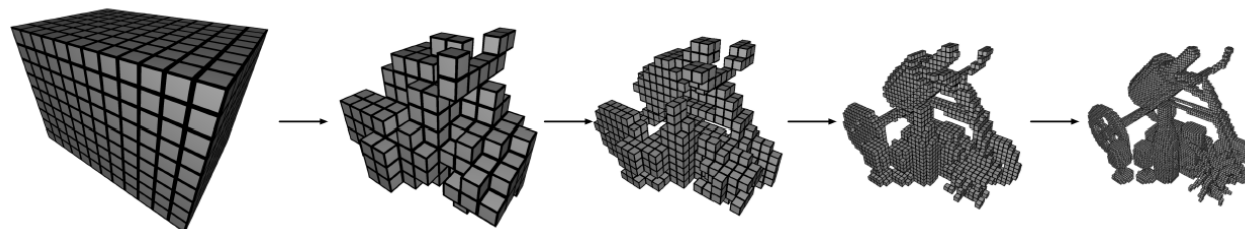
Illustration of NSVF

Our Method (NSVF)

- Scene Representation - Neural Sparse Voxel Fields (NSVF).
- Volume Rendering with NSVF.



- Progressive Learning: we train NSVF progressively with the differentiable volume rendering operation from a set of posed 2D images.



Scene Representation - NSVF

The scene is modeled as a set of voxel-bounded implicit functions:

$$F_{\theta}(\mathbf{p}, \mathbf{v})$$

The relevant non-empty parts of a scene are contained within a set of sparse bounding voxels:

$$\mathcal{V} = \{V_1 \dots V_K\}$$

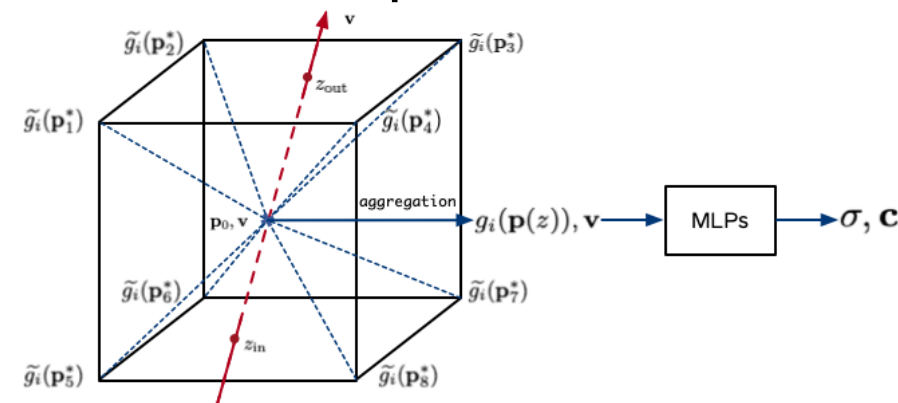
Scene Representation - NSVF

A voxel-bounded implicit field

- For a given point $\mathbf{p}$ inside voxel V_i , the voxel-bounded implicit field is defined as:

$$F_{\theta}^i : (g_i(\mathbf{p}), \mathbf{v}) \rightarrow (\mathbf{c}, \sigma), \forall \mathbf{p} \in V_i,$$

$g_i(\mathbf{p})$: voxel embedding
 $\mathbf{v}$: ray direction
 $\mathbf{c}$: color
 σ : density



- Voxel embedding is defined as:

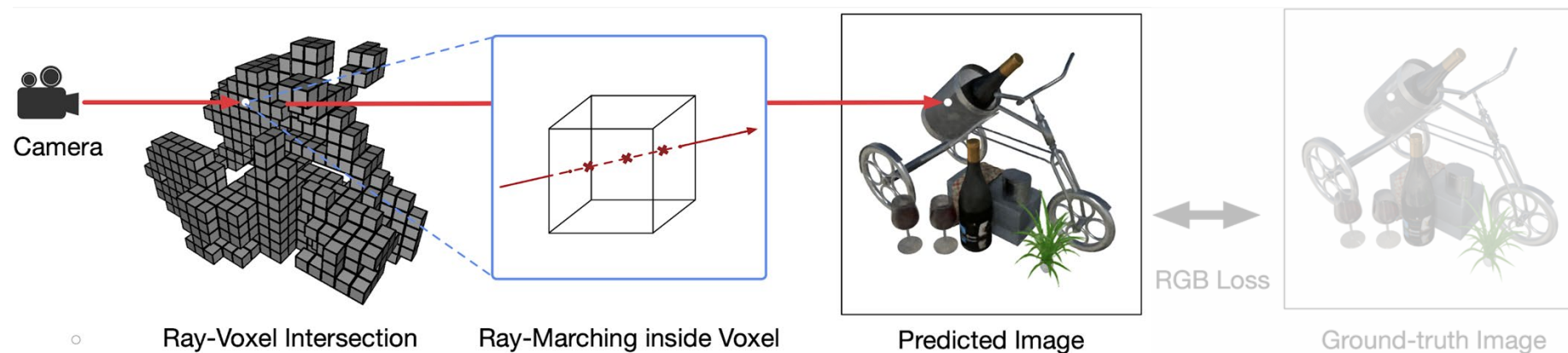
$$g_i(\mathbf{p}) = \zeta(\chi(\tilde{g}_i(\mathbf{p}_1^*), \dots, \tilde{g}_i(\mathbf{p}_8^*)))$$

ζ : Positional encoding
 χ : Trilinear interpolation
 $\tilde{g}_i(\mathbf{p}_j^*)$: Voxel features (e.g. learnable voxel embeddings)

Volume Rendering with NSVF

Rendering NSVF is fast because it avoids sampling points in the empty space.

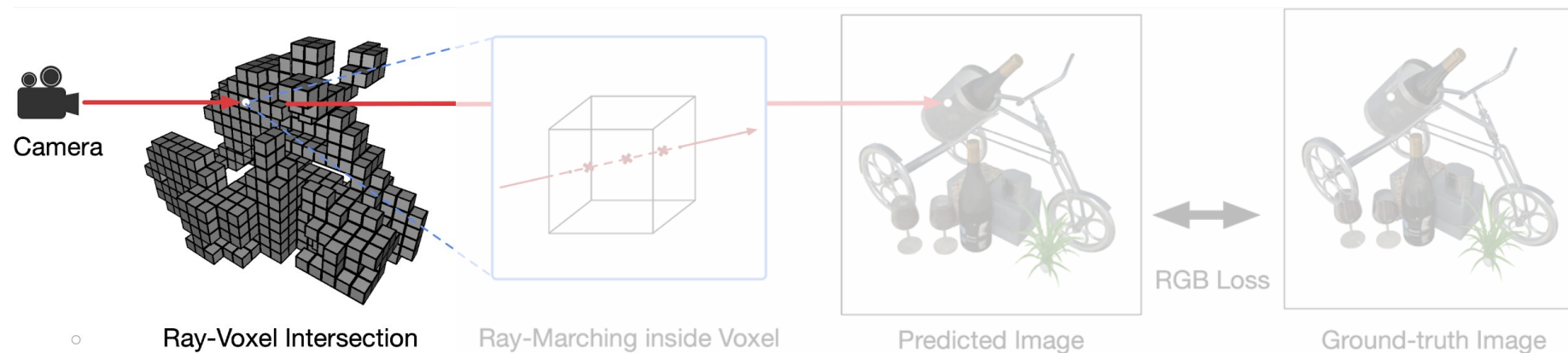
- Ray-voxel Intersection.
- Ray marching inside voxels.



Volume Rendering with NSVF

Ray-voxel Intersection

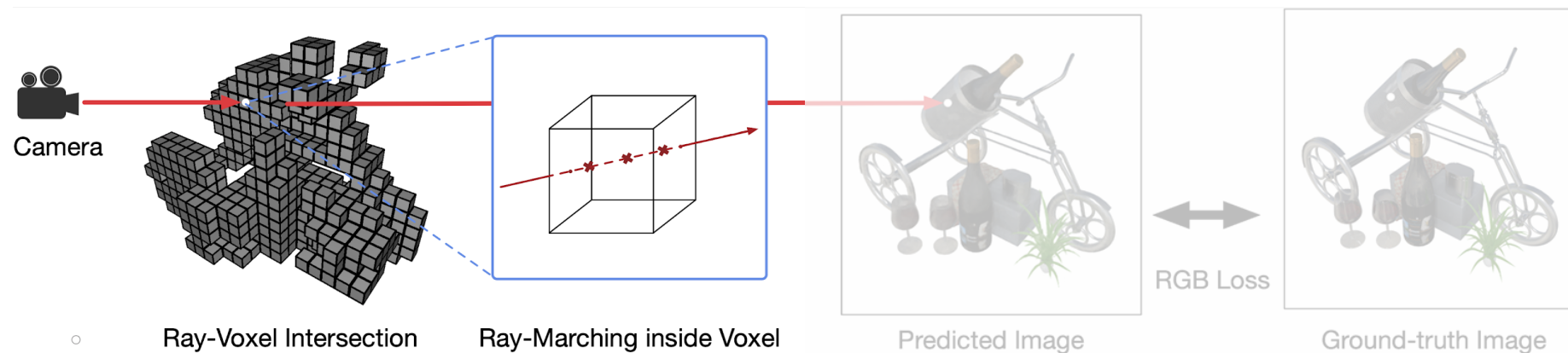
- Apply Axis Aligned Bounding Box (AABB) intersection test [Haines, 1989] for each ray.
- AABB is very efficient for NSVF, handling millions of ray-voxel intersections in real time.



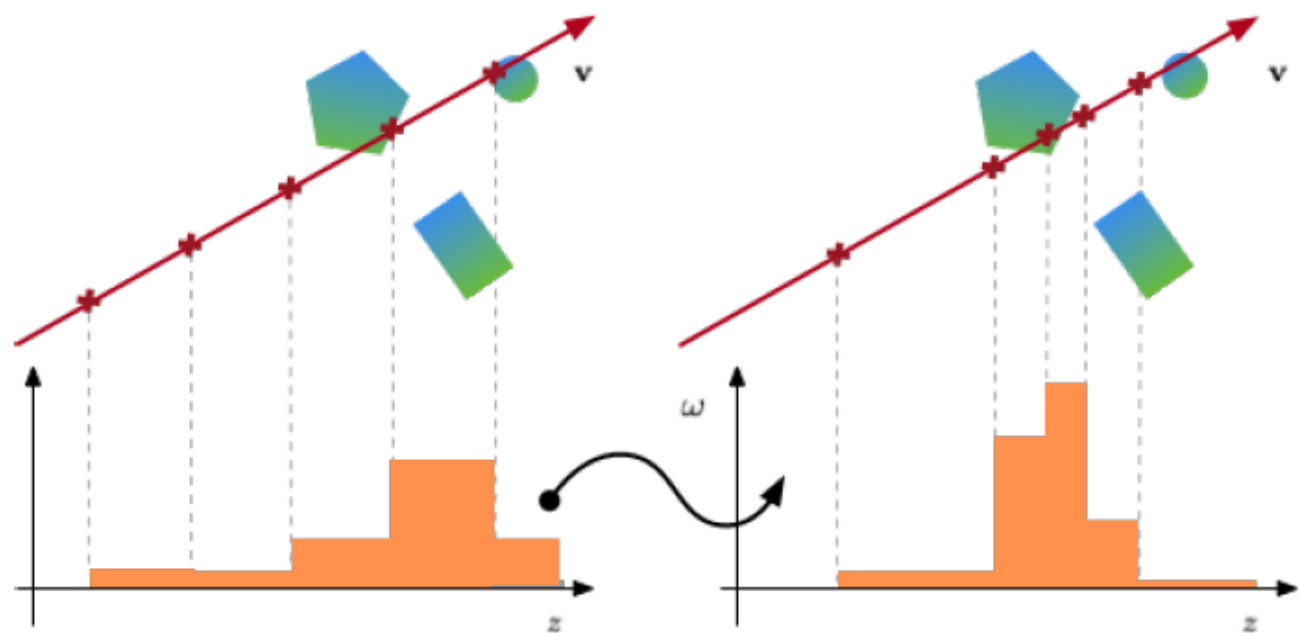
Volume Rendering with NSVF

Ray Marching inside Voxels

- Uniformly sample points along the ray inside each intersected voxel, and evaluate NSVF to get the color and density of each sampled point.



Volume Rendering with NSVF



Coarse sampling

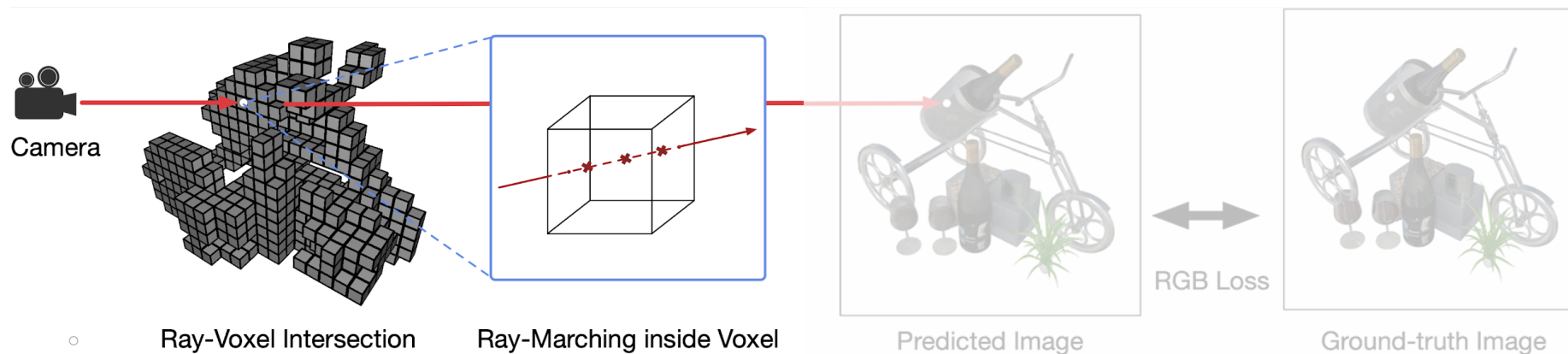
Importance sampling

NeRF's sampling method

Volume Rendering with NSVF

Early Termination

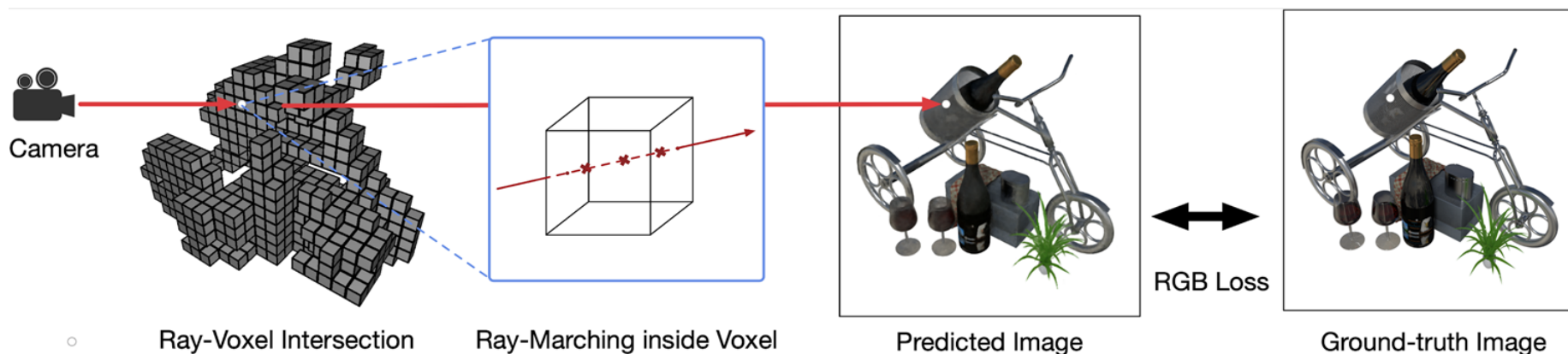
- Avoid taking unnecessary accumulation steps behind the surface;
- Stop evaluating points earlier when the accumulated densities close to 1



Progressive Learning

- Because our rendering process is differentiable, the model can be trained end-to-end with 2D posed images as input for supervision.

$$\mathcal{L} = \sum_{(p_0, v) \in R} \underbrace{\|C(p_0, v) - C^*(p_0, v)\|_2^2}_{\text{Predicted color} \quad \text{Ground truth color}} + \lambda \cdot \Omega(A(p_0, v))$$



Progressive Learning

A progressive training strategy to learn NSVF from coarse to fine

- Voxel Initialization
- Self-Pruning
- Progressive Training

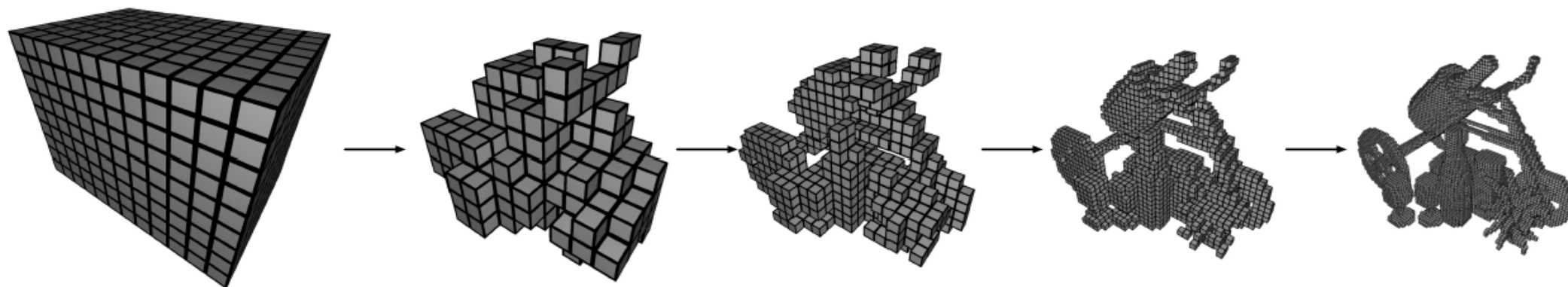
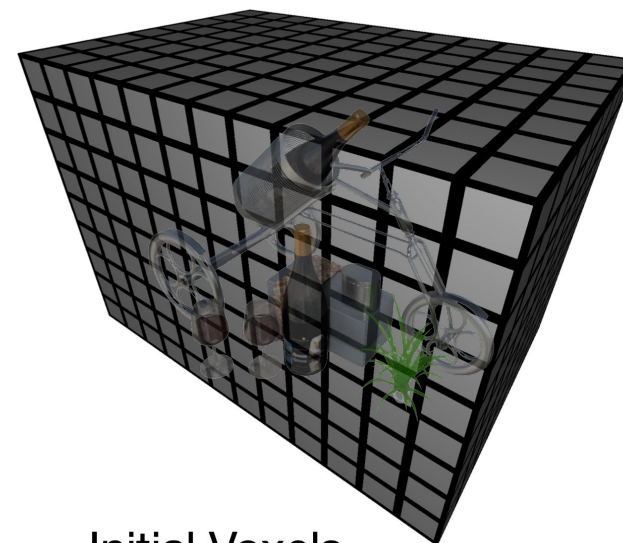


Illustration of self-pruning and progressive training

Progressive Learning

Voxel Initialization

- The initial bounding box encloses the whole scene with sufficient margin. We eventually subdivide the bounding box into ~ 1000 voxels.
- If a coarse geometry is available, the initial voxels can also be initialized by voxelizing the coarse geometry.



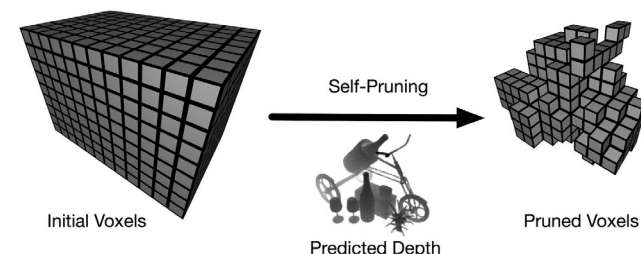
Initial Voxels

Progressive Learning

Self-Pruning

- We improve rendering efficiency by pruning “empty” voxels.
 - Determine whether a voxel is empty or not by checking the maximum predicted density from sampled points inside the voxel.

$$V_i \text{ is pruned if } \min_{j=1\dots G} \exp(-\sigma(\underbrace{g_i(\mathbf{p}_j)}_{\text{density}})) > \gamma, \quad \mathbf{p}_j \in V_i, V_i \in \mathcal{V},$$



- Since this pruning process does not rely on other processing modules or input cues, we call it “*self-pruning*”.

Progressive Learning

Progressive Training

- Self-pruning enables us to progressively allocate our resources.
- Progressive training:
 - Halve the size of voxels $\rightarrow$ Split each voxel into 8 sub-voxels.
 - Halve the size of ray marching steps.
 - The feature representations of the new vertices are initialized via trilinear interpolation of feature representations at the original eight voxel vertices.

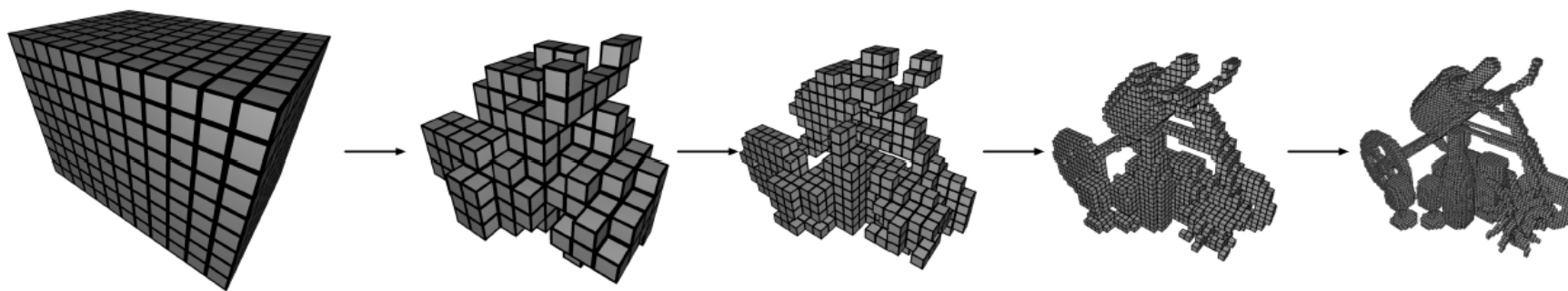


Illustration of self-pruning and progressive training

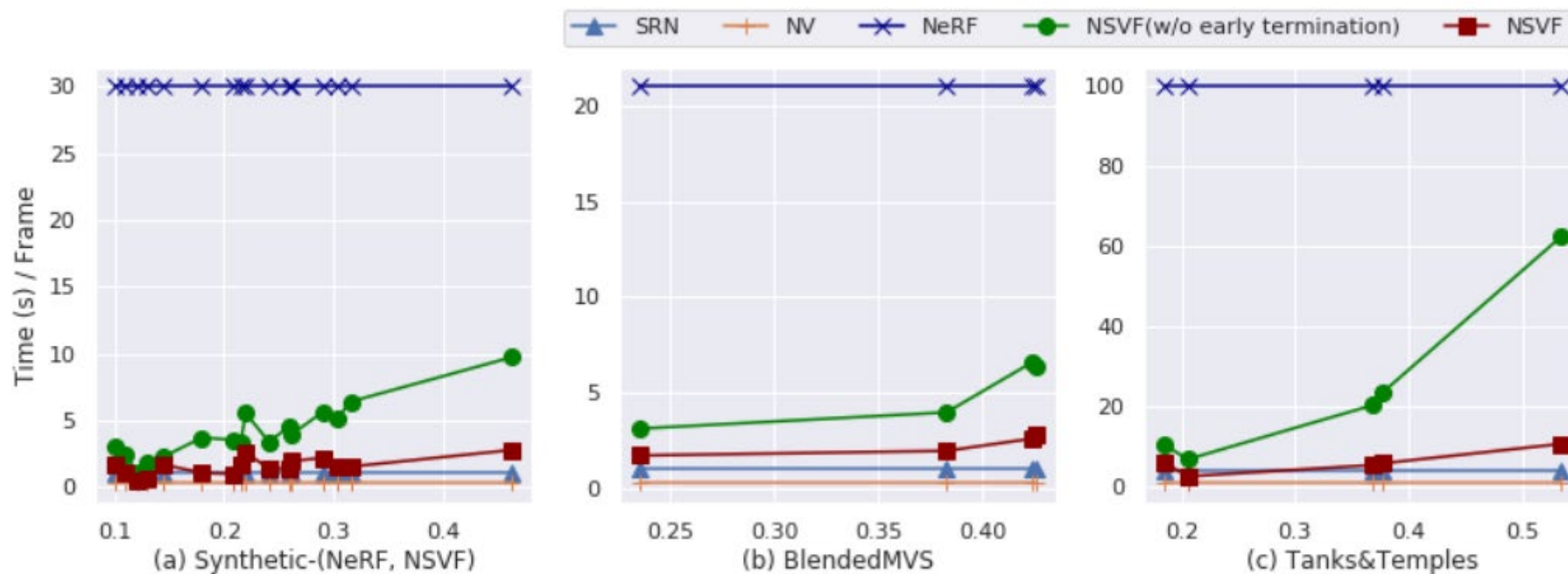
Results

Models	Synthetic-NeRF			Synthetic-NSVF			BlendedMVS			Tanks and Temples		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
SRN	22.26	0.846	0.170	24.33	0.882	0.141	20.51	0.770	0.294	24.10	0.847	0.251
NV	26.05	0.893	0.160	25.83	0.892	0.124	23.03	0.793	0.243	23.70	0.834	0.260
NeRF	31.01	0.947	0.081	30.81	0.952	0.043	24.15	0.828	0.192	25.78	0.864	0.198
NSVF ⁰	31.75	0.954	0.048	35.18	0.979	0.015	26.89	0.898	0.114	28.48	0.901	0.155
NSVF	31.74	0.953	0.047	35.13	0.979	0.015	26.90	0.898	0.113	28.40	0.900	0.153

*NSVF⁰ is without early termination

*NSVF is executed with early termination ($\varepsilon = 0.01$)

Results



x-axis: foreground to background ratio
 y-axis: rendering time in second

*NSVF⁰ is without early termination (**Green curve**)

*NSVF is executed with early termination ($\varepsilon = 0.01$) (**Red curve**)

Results

Robot (From SyntheticNSVF dataset)

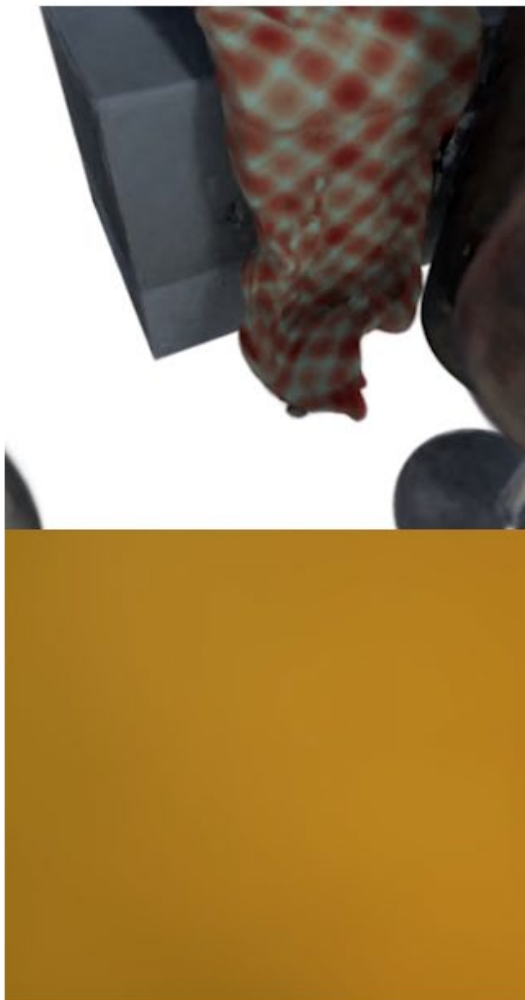


NeRF (Mildenhall et al. 2020)
(Rendering speed: 30 s/frame)

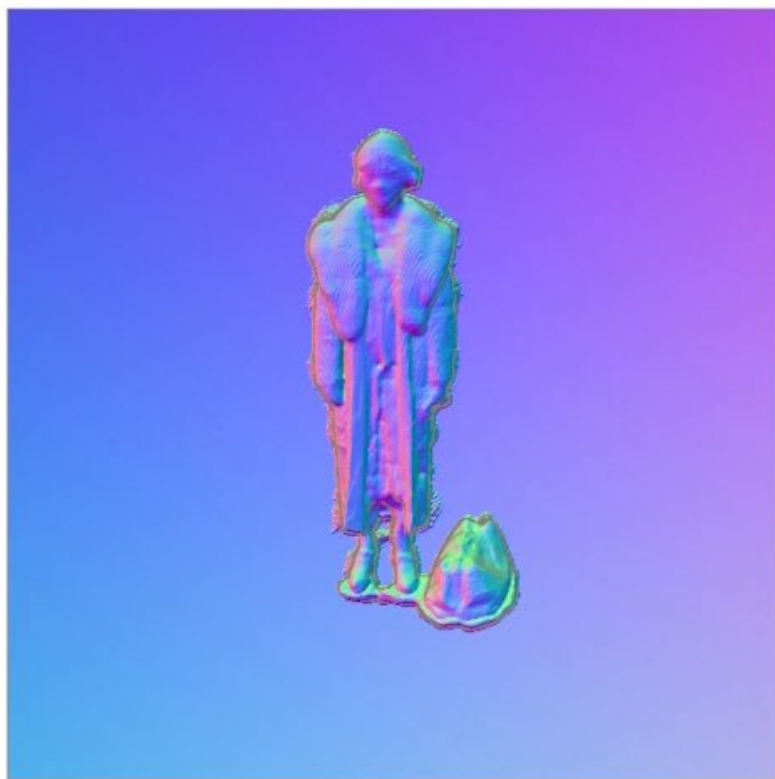


Ours (NSVF)
(Rendering speed: 0.6 s/frame)

Zoom-in & Zoom-out Effects



Rendering of Dynamic Scenes



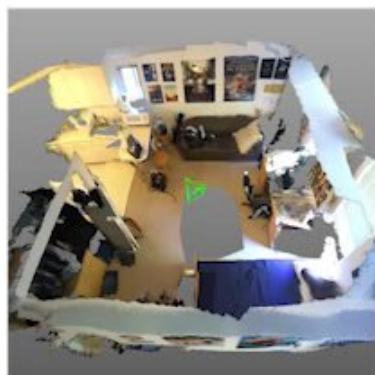
Normals of NSVF result



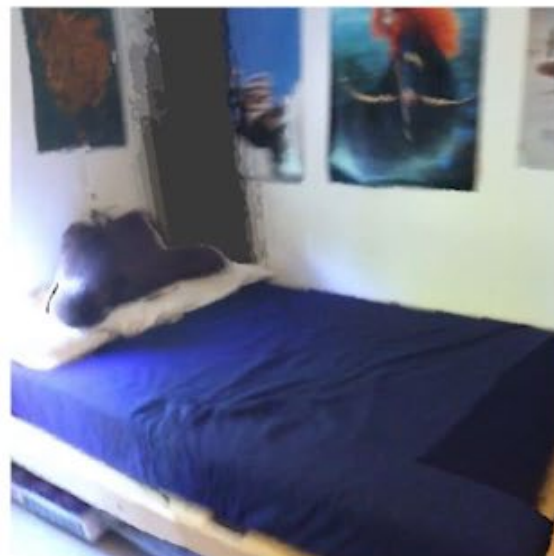
NSVF

(Input sequence from Fraunhofer Heinrich Hertz Institute)

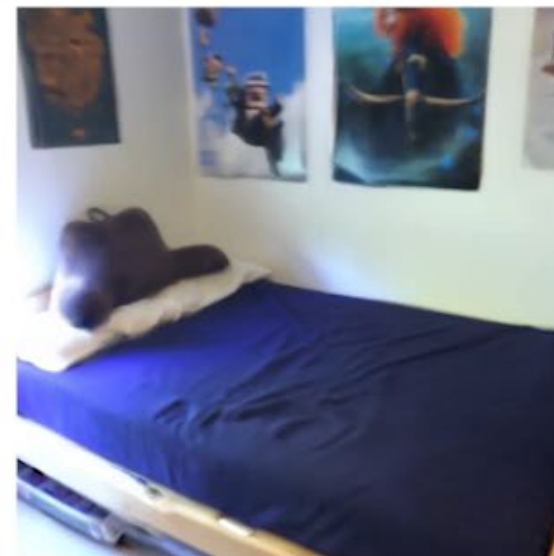
Rendering of Large-scale Indoor Scenes



Interactive
camera control

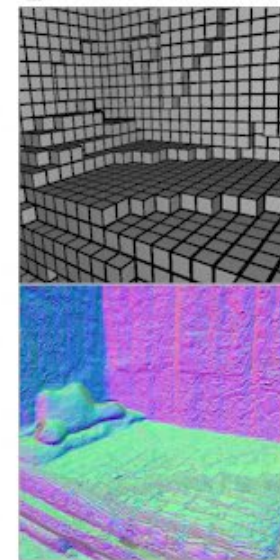


Users' view
(Rendered mesh)



NSVF

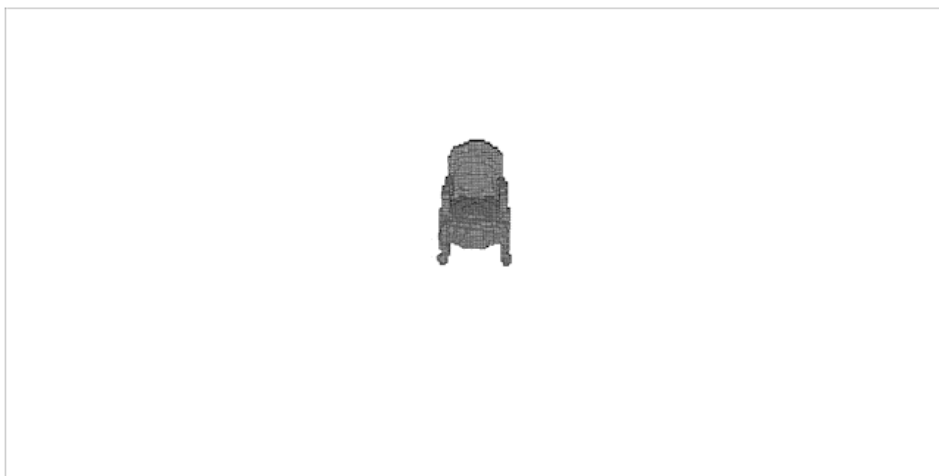
Sparse voxels



Normals

(Input sequence and 3D mesh from ScanNet [Dai et al. 2017])

Scene Editing and Composition



Interactive editing

NSVF

Main Limitation

- Real-time performance
 - Although our method is typically 10x faster than Nerf, it is still far from real time performance.
 - NeRF **0.06 FPS** v.s. NSVF **1.1 FPS** v.s. Real-time Rendering **>25 FPS**

Real-time NeRF Rendering

Caching the Network Outputs with a Sparse Voxel Octree.

- The key idea is to use caching to trade memory for computational efficiency at inference time.

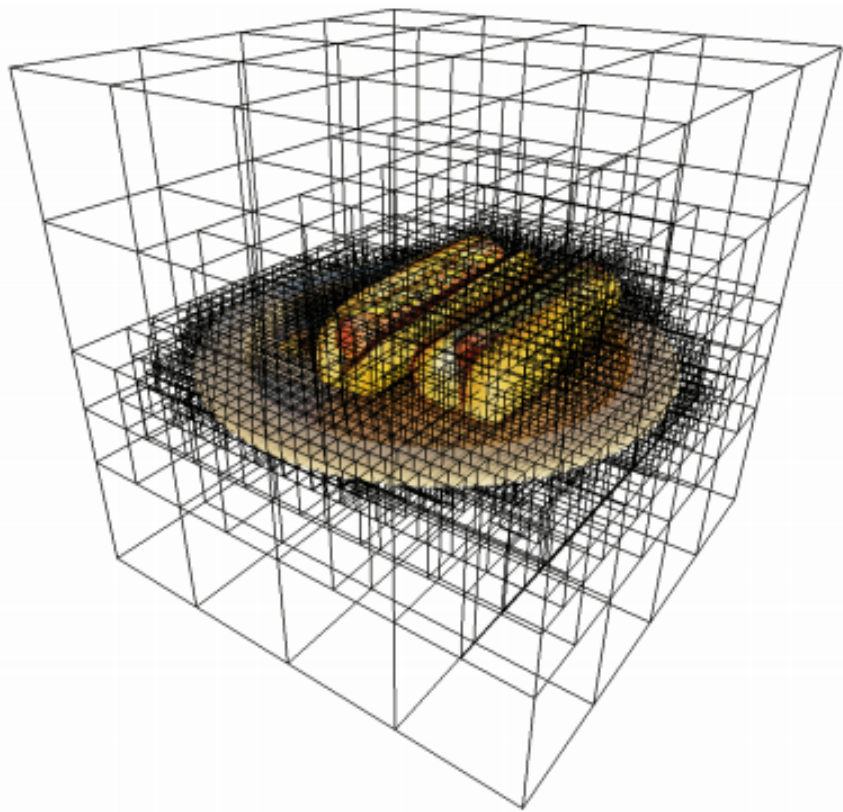


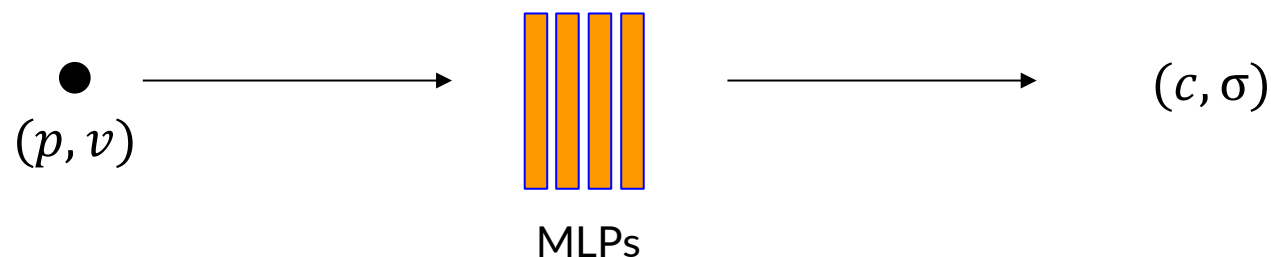
Image from [Yu et al., 2021]

Caching the Network Outputs with a Sparse Voxel Octree.

- The key idea is to use caching to trade memory for computational efficiency at inference time.
- There are three related papers:
- PlenOctrees for Real-time Rendering of Neural Radiance Fields, Yu et al., Arxiv 2021 ~200FPS
- FastNeRF: High-Fidelity Neural Rendering at 200FPS, Garbin et al., Arxiv 2021 ~200FPS
- Baking Neural Radiance Fields for Real-Time View Synthesis, Hedman et al., Arxiv 2021 ~84FPS

Caching the Network Outputs with a Sparse Voxel Octree.

- The key idea is to use caching to trade memory for computational efficiency at inference time.
- Specifically,
 - (1) Train a NeRF-like network to predict density and color for each sampled point.



Caching the Network Outputs with a Sparse Voxel Octree.

- The key idea is to use caching to trade memory for computational efficiency at inference time.
- Specifically,
 - (1) Train a NeRF-like network to predict density and color for each sampled point.
 - (2) After training, extract the volumetric content and represent it using a sparse voxel Octree.
 - (3) Precompute the network outputs for each octree leaf.

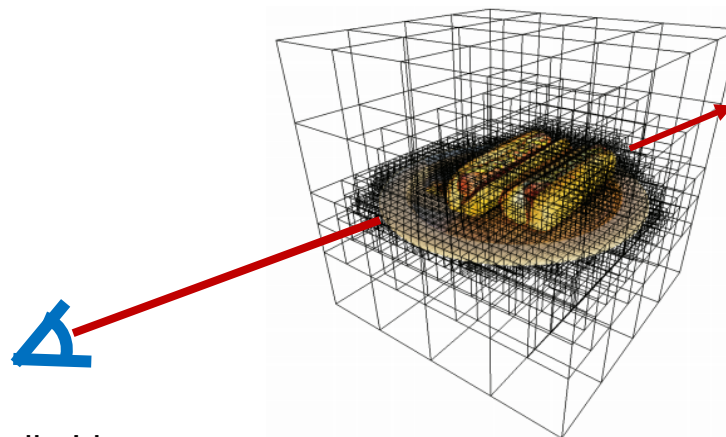


Image from [Yu et al., 2021]

Using Multiple Shallow Networks

- A single high-capacity MLP for representing the entire scene can be replaced with thousands of small MLPs for the decomposed parts of the scene.
- KiloNeRF: Speeding up Neural Radiance Fields with Thousands of Tiny MLPs, Reiser et al., Arxiv 2021 **~13 FPS**

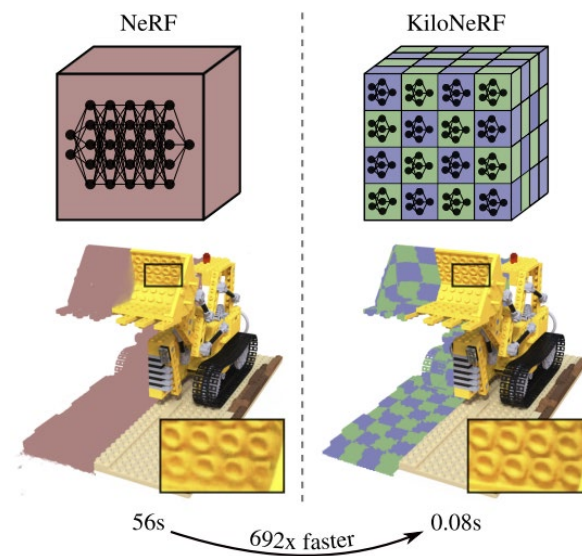


Image from [Reiser et al., 2021]

Using Multiple Shallow Networks

- A single high-capacity MLP for representing the entire scene can be replaced with thousands of small MLPs for the decomposed parts of the scene.
- The similar idea is also used in:

DeRF: Decomposed Radiance Fields, Rebain et al., CVPR 2021 **~0.18 FPS**



Image from [Rebain et al., 2021]

Mixture of Volumetric Primitives

Lombardi, Simon, Schwartz, Zollhoefer, Sheikh, Saragih

SIGGRAPH 2021

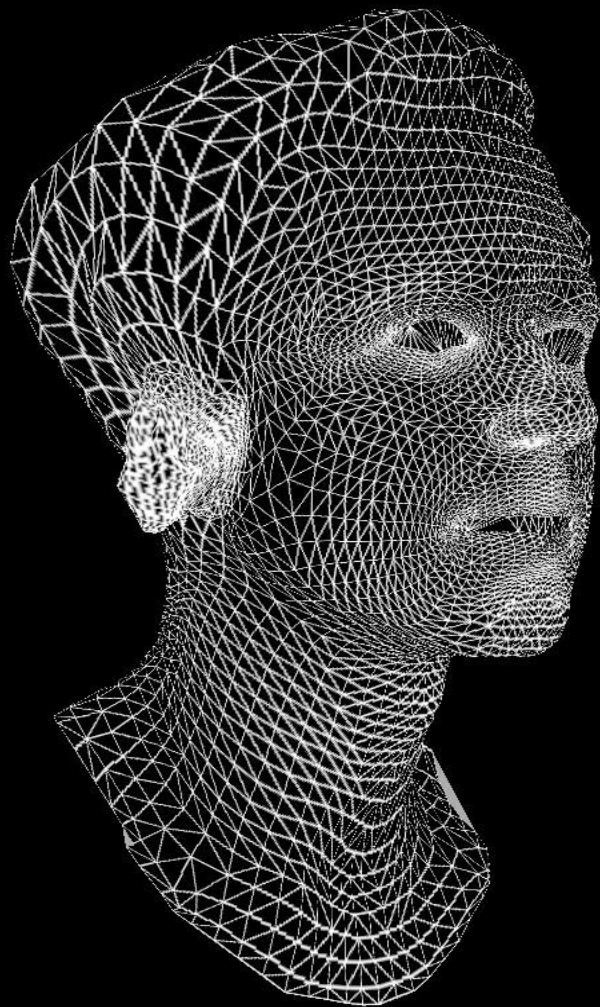
Mesh-based Rendering

Expression
vector



Camera
pose

Decoder



Mesh vertices



Texture

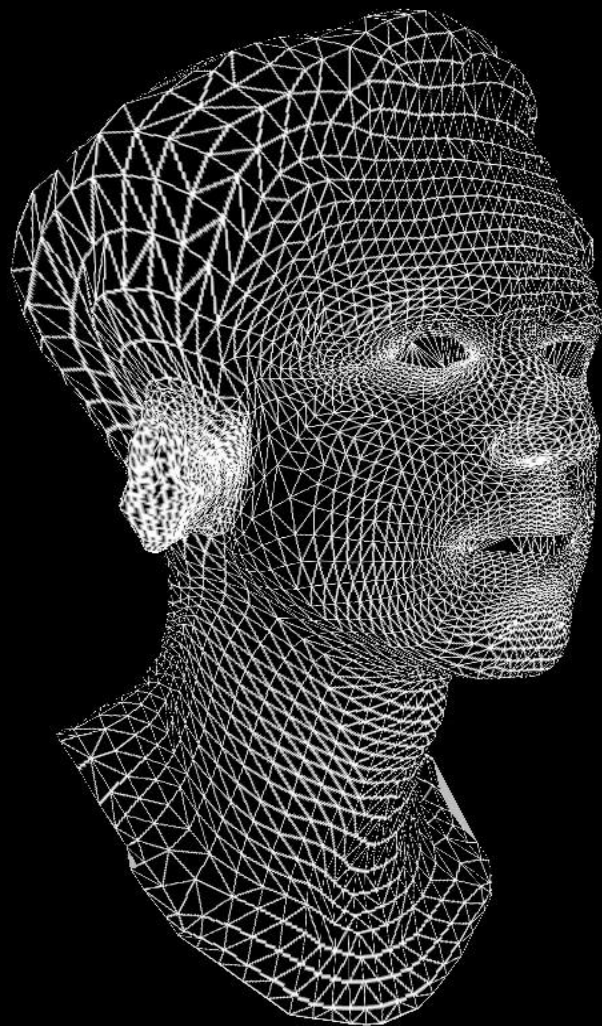
Mixture of Volumetric Primitives

Expression
vector



Camera
pose

Decoder



Mesh vertices

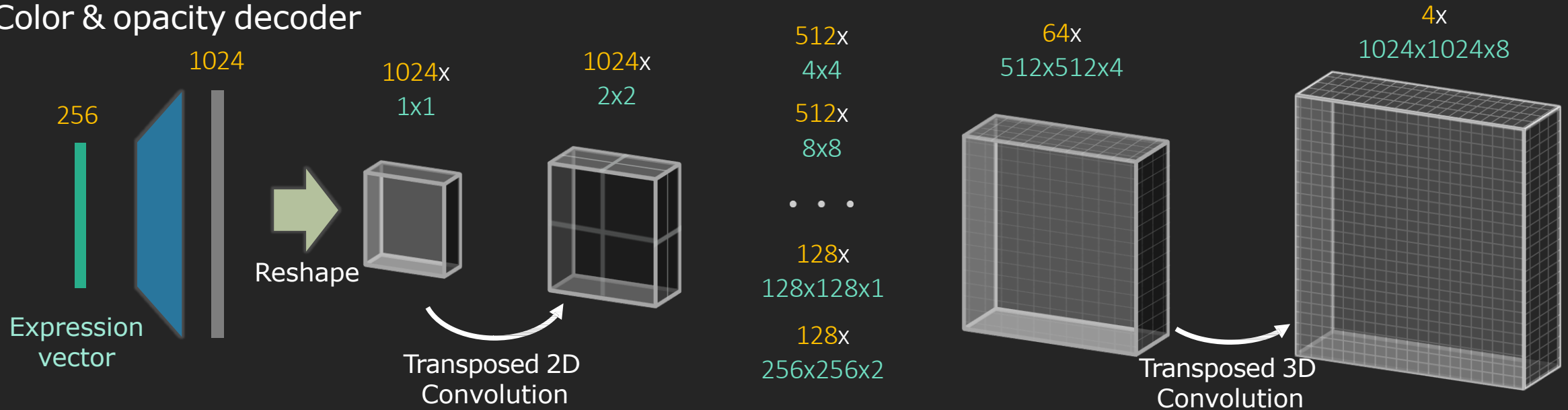


Volumetric
"texture"

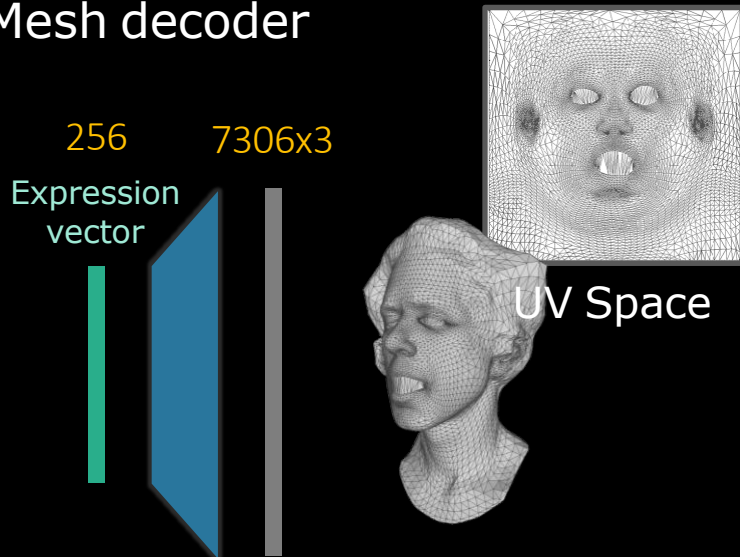
MVP Decoder

Features x
Spatial Dimensions

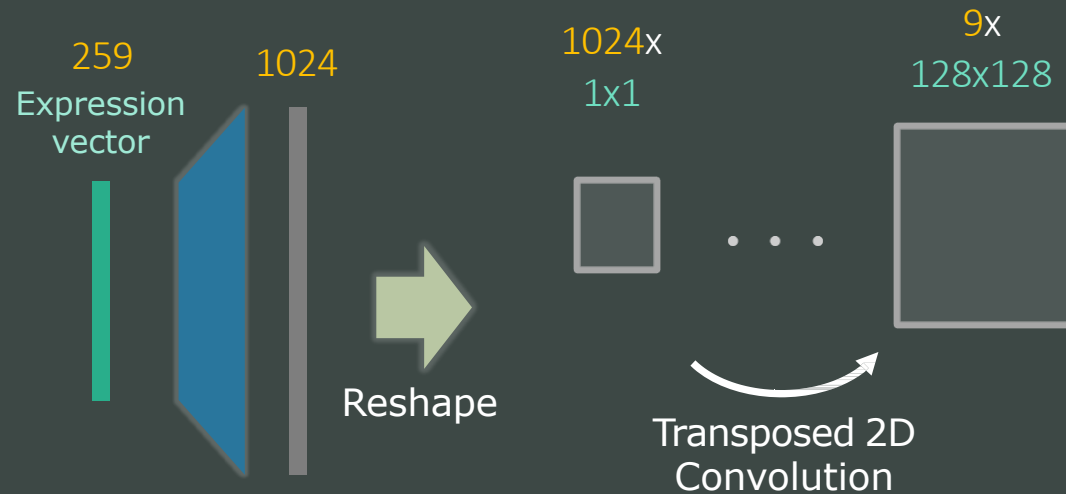
Color & opacity decoder



Mesh decoder



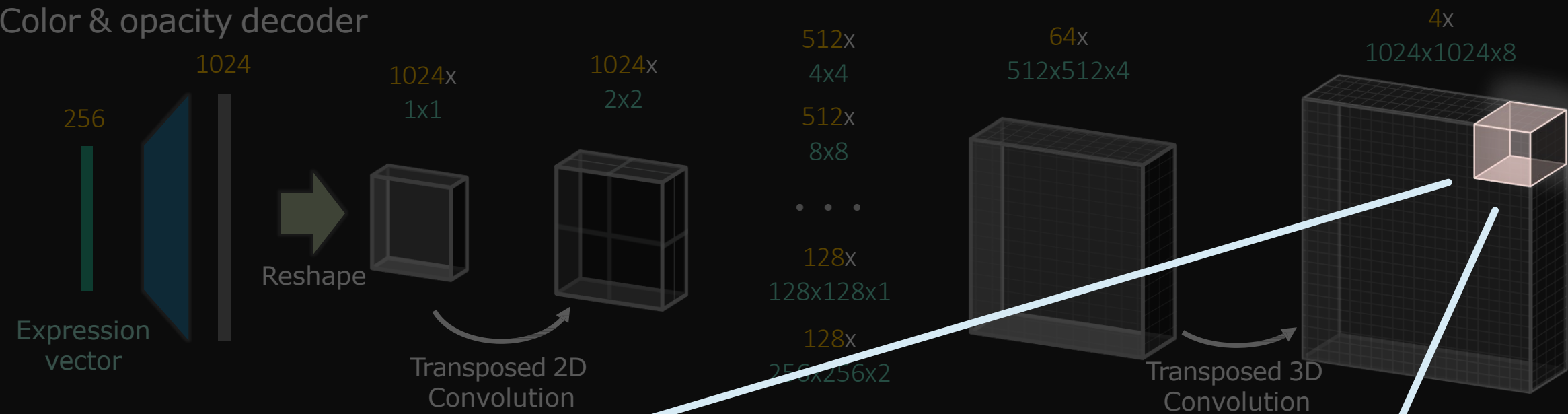
Motion decoder



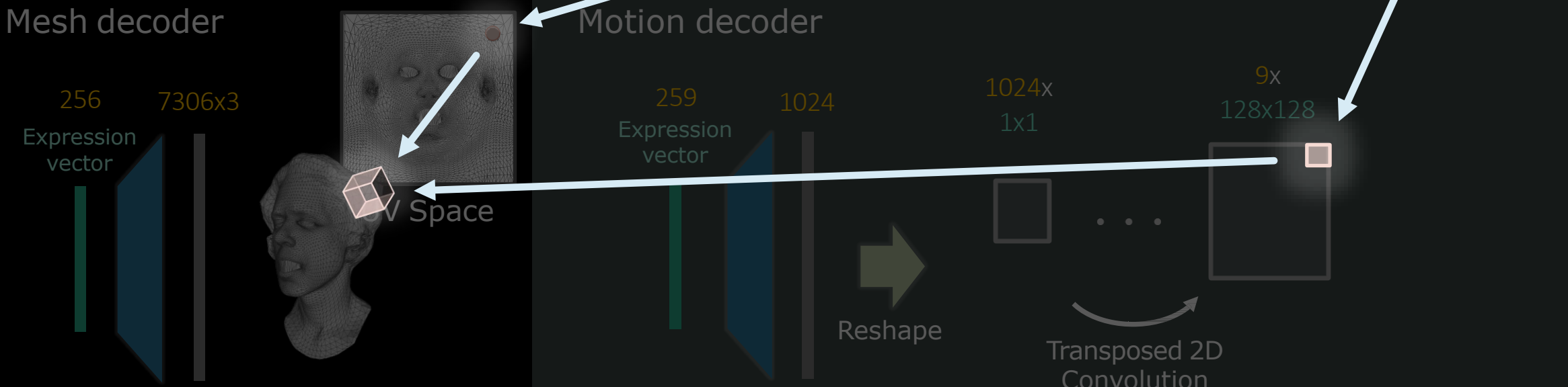
MVP Decoder

Features x
Spatial Dimensions

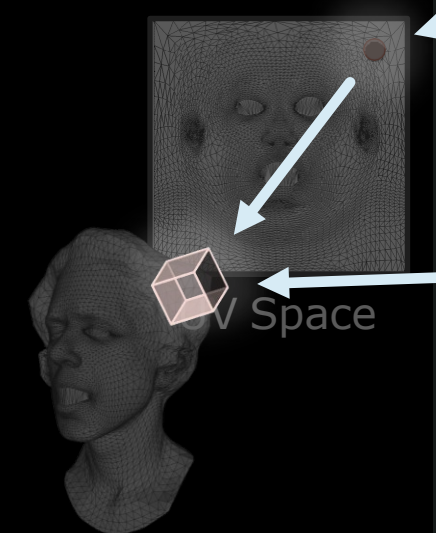
Color & opacity decoder



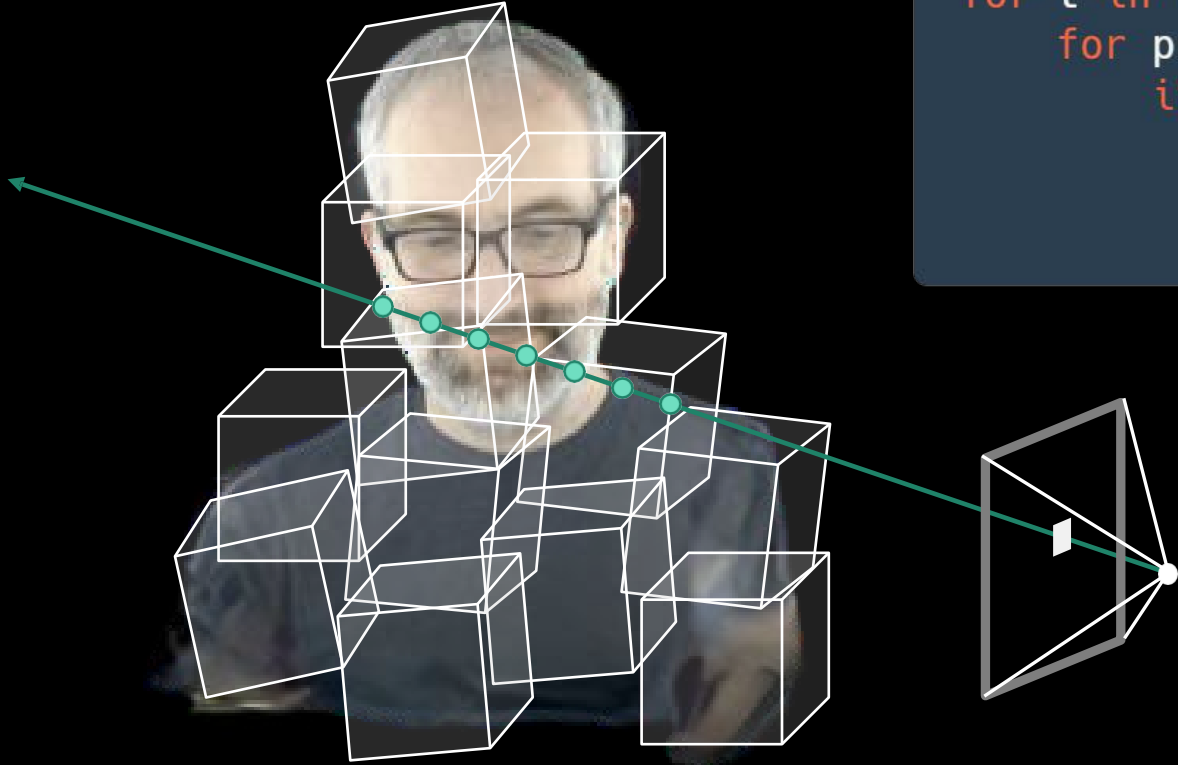
Mesh decoder



Motion decoder



Raymarching MVP



```
for t in range(t_min, t_max, step_size):  
    for primitive in primitive_list:  
        if raystart + t * raydir in primitive:  
            rgb, alpha = sample_rgba(primitive, t)  
            aggregate_radiance(rgb, alpha)
```

Results



Depth-guided Sampling

- Predicting depths for more efficient sampling:

DONeRF: Towards Real-Time Rendering of Neural Radiance Fields using Depth Oracle Networks, Neff et al., Arxiv 2021 ~15 FPS

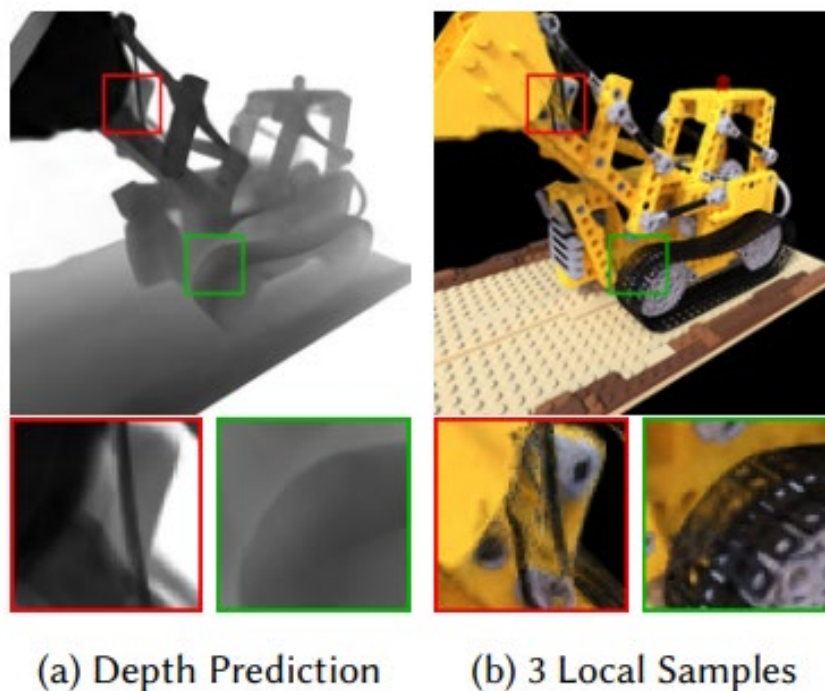


Image from [Neff et al. 2021]

Learning Integral by a Neural Network

- A general framework to integrate signals with implicit neural representation, which can be used in volume rendering.

AutoInt: Automatic Integration for Fast Neural Volume Rendering, Lindell et al., CVPR 2021. **~0.4 FPS**

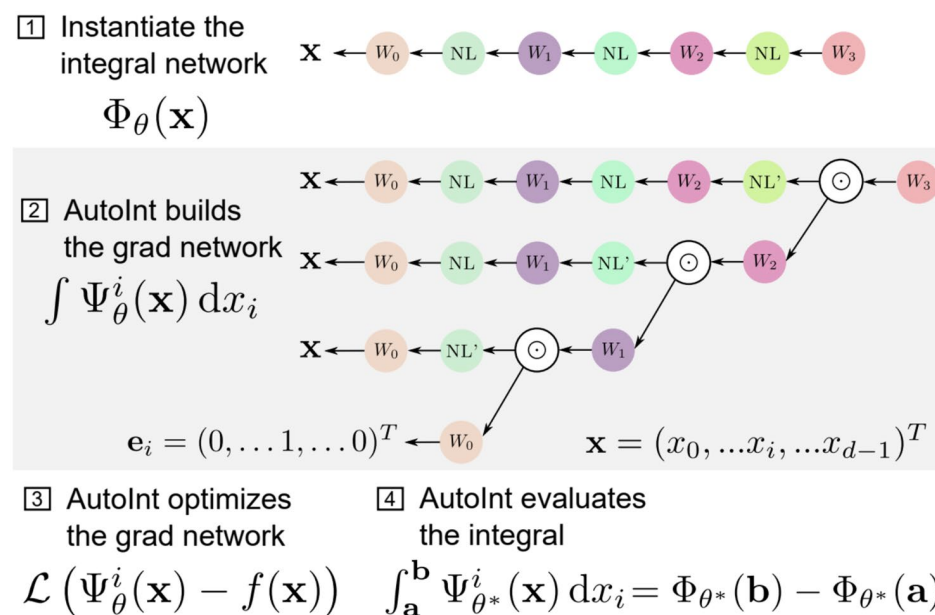
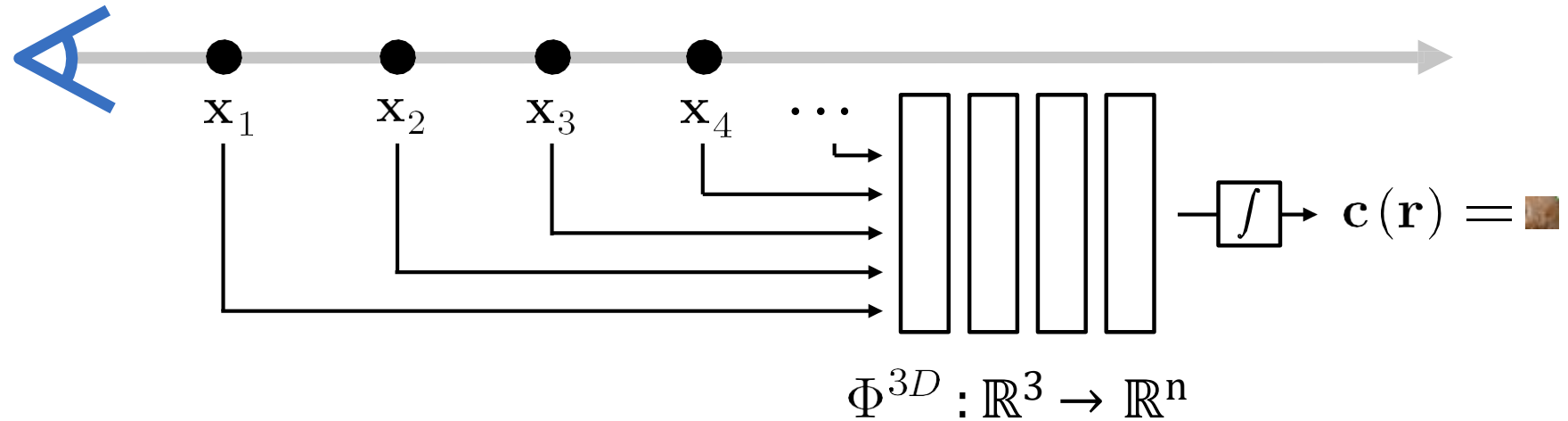
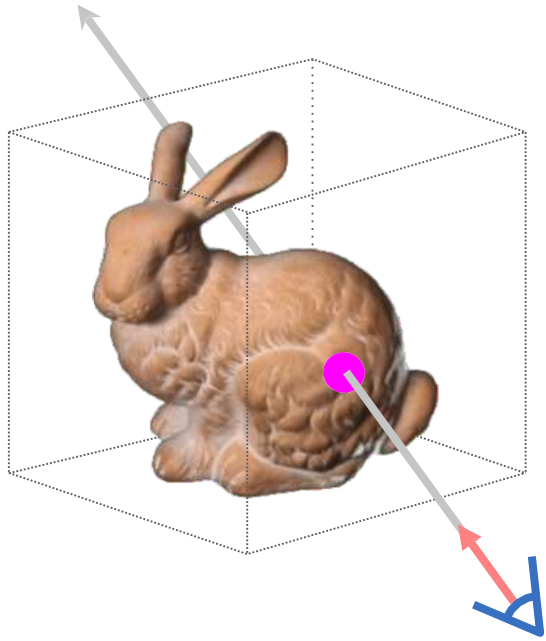


Image from [Lindell et al. 2021]

3D-structured Neural Scene Representations

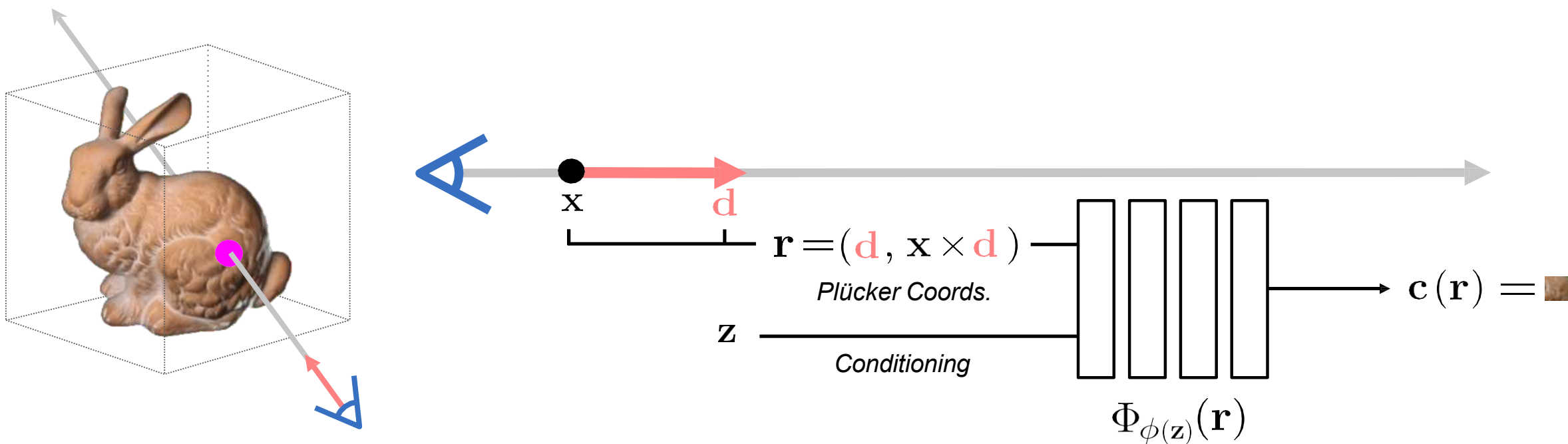


Hundreds of samples per ray.

256x256 image takes >30 seconds (volumetric).

Time- and memory-intensive training.

Light Field Networks



An Alternative Scene Representation

Real-time. No post-processing, no discrete data structures (octrees, voxelgrids, ...).
 >100x reduction in memory: Can be trained on small GPUs!

Related Work

- NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis, Mildenhall et al., ECCV 2020
- Neural Sparse Voxel Fields, Liu et al., NeurIPS 2020
- AutoInt: Automatic Integration for Fast Neural Volume Rendering, Lindell et al., CVPR 2021
- DeRF: Decomposed Radiance Fields, Rebain et al., CVPR 2021
- DOnERF: Towards Real-Time Rendering of Neural Radiance Fields using Depth Oracle Networks, Neff et al., Arxiv 2021
- FastNeRF: High-Fidelity Neural Rendering at 200FPS, Garbin et al., Arxiv 2021
- KiloNeRF: Speeding up Neural Radiance Fields with Thousands of Tiny MLPs, Reiser et al., Arxiv 2021
- PlenOctrees for Real-time Rendering of Neural Radiance Fields, Yu et al., Arxiv 2021
- Baking Neural Radiance Fields for Real-Time View Synthesis, Hedman et al., Arxiv 2021
- NeX: Real-time View Synthesis with Neural Basis Expansion. Wizaradwongsa et al., CVPR 2021

Related Work

- Mixture of Volumetric Primitives, Lombodi et al., SIGGRAPH 2021
- Light Field Networks: Neural Scene Representations with Single-Evaluation Rendering, Sitzmann et al., NeurIPS 2021

Acknowledgments

- Advances in Neural Rendering
- Neural Fields in Visual Computing and Beyond
- awesome-NeRF: a curated list of awesome neural radiance fields papers
- MPII Summer Semester 2023: Computer Vision and Machine Learning for Computer Graphics
- Vincent Sitzmann's slides for LFNs, Stephen Lombardi's slides for Mixture of Volumetric Primitives

Any Questions?